

RESEARCH PAPER — SYSTEM ARCHITECTURE

M.A.C.: A Real-Time Modular Architecture for an Autonomous Minecraft Agent

Integrating JARVIS Cognition, Quantized LLM Reasoning, Tri-Partite Memory, and Baritone Pathfinding

M.A.C. Research Team
April 2026

This paper presents the architectural design and preliminary evaluation of M.A.C. (Minecraft Autonomous Companion), a modular single-agent system for real-time autonomous operation within Minecraft. The system integrates a four-stage JARVIS cognitive loop, a tri-partite memory system, a quantized 70B-parameter LLM reasoning engine, and the Baritone pathfinding engine. We position this work as a system paper establishing architectural foundations and experimental protocols for rigorous future validation.

Contents

Abstract	3
1 Introduction	3
2 Related Work	4
2.1 LLM-Powered Embodied Agents	4
2.2 Memory Systems for Autonomous Agents	5
2.3 Pathfinding in Minecraft	5
3 System Architecture	5
3.1 Perception Layer	6
3.2 Tri-Partite Memory System	6
3.2.1 Episodic Memory (Zep Cloud)	6
3.2.2 Semantic Memory	6
3.2.3 Spatial Memory (GraphRAG)	7
3.3 Reasoning Engine	7
3.3.1 Model Selection and Quantization	7
3.3.2 Inference Infrastructure	7
3.3.3 Context Assembly	7
3.4 Baritone Pathfinding System	8
3.5 Action Execution Layer	8
4 Decision Cycle Pipeline	9
4.1 Cycle Timing	9
4.2 Interrupt Handling	9
4.3 Asynchronous Memory Storage	9
5 Integration Architecture	9
5.1 System Topology	10
5.2 Data Flow	10
5.3 Component Communication	10
5.4 Hardware and Deployment	10
6 Experimental Framework	10
6.1 Experimental Setup	11
6.2 Task Protocols	11
6.3 Component Benchmarks	12
6.4 Preliminary System Performance	12
6.5 Behavioral Scenario Results	12
7 Ablation Study Framework	14
7.1 Proposed Ablation Conditions	14
7.2 Ablation Implementation	14
8 Baseline Comparison Framework	15
8.1 Tier 1: Scripted Baseline	15
8.2 Tier 2: LLM-Only Baseline	15
8.3 Tier 3: Literature Comparison	15

9	Discussion	15
9.1	Architectural Contributions	15
9.2	Limitations and Threats to Validity	16
9.3	Comparison with Related Systems	16
10	Reproducibility	16
11	Conclusion and Future Work	17

Abstract

This paper presents M.A.C. (Minecraft Autonomous Companion), a modular single-agent architecture for real-time autonomous operation within Minecraft. The system integrates a four-stage JARVIS cognitive loop—Perception, Memory, Reasoning, and Action—that bridges high-level large language model (LLM) reasoning with low-level motor control via specialized algorithmic pathfinding. M.A.C. employs a tri-partite memory system comprising episodic memory (Zep Cloud), semantic knowledge, and spatial memory (GraphRAG), alongside the Baritone pathfinding engine for terrain navigation and an execution bridge using RCON and WebSocket protocols. The reasoning core utilizes a quantized DeepSeek-R1-70B-AWQ model deployed on an AMD MI300X GPU through a vLLM inference server. We describe the complete system architecture, detail the integration mechanisms between components, and present a preliminary evaluation framework for validating the design. We position this work as a system paper that establishes the architectural foundations and identifies the experimental protocols necessary for rigorous empirical validation in future work, rather than claiming definitive performance results at this stage.

Keywords: autonomous agents, Minecraft, large language models, JARVIS architecture, real-time decision-making, modular cognition, GraphRAG, Baritone pathfinding

1 Introduction

The development of autonomous agents capable of sustained, goal-directed behavior in complex, open-ended three-dimensional environments remains one of the defining challenges in artificial intelligence research. Minecraft, as a procedurally generated sandbox world with deep crafting trees, dynamic threat landscapes, and no predefined narrative trajectory, has emerged as a particularly compelling testbed for embodied intelligence. Unlike constrained game environments where the state space is bounded and the action repertoire is finite, Minecraft demands that an agent continuously perceive an ever-changing environment, maintain and retrieve knowledge across extended time horizons, reason about multi-step objectives under uncertainty, and translate abstract plans into precise motor actions—all in real time.

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding, multi-step reasoning, and plan generation, making them attractive candidates for the cognitive core of autonomous agents. However, deploying LLMs as the reasoning engine for an embodied agent introduces several fundamental challenges that prior work has addressed to varying degrees. First, LLMs operate on discrete token sequences and cannot natively process the continuous, high-dimensional sensory streams that an agent in a 3D world encounters. Second, LLMs have finite context windows that are insufficient for maintaining the long-term spatial and episodic knowledge that autonomous exploration requires. Third, the latency of LLM inference can violate the real-time constraints of interactive environments where threats emerge and opportunities vanish on sub-second timescales. Fourth, LLMs lack the fine-grained motor control necessary for navigating complex terrain, mining blocks, and executing precise crafting sequences.

These challenges motivate a *modular* architectural approach, in which specialized subsystems handle distinct cognitive functions—perception, memory, reasoning, and action—and communicate through well-defined interfaces. The key insight behind M.A.C. is that by decoupling high-level cognitive reasoning from low-level spatial navigation and motor control, and by maintaining a multi-tiered persistent memory system, an LLM-based agent can achieve real-time responsiveness and long-term goal execution without requiring the LLM to handle every aspect of cognition simultaneously.

This paper makes the following contributions:

1. We present the complete architectural design of M.A.C., a single-agent system that integrates a four-stage JARVIS cognitive loop with a tri-partite memory system and the Baritone pathfinding engine for real-time autonomous operation in Minecraft.
2. We detail the integration mechanisms—WebSocket, RCON, and a command translation bridge—that connect LLM reasoning to Minecraft-native execution, enabling the system to operate within a 200 ms decision cycle budget.
3. We describe the deployment of a quantized DeepSeek-R1-70B-AWQ model on an AMD MI300X GPU using the vLLM inference server, and characterize the design choices around quantization, context management, and hardware selection.
4. We present a preliminary evaluation of the integrated system across targeted behavioral scenarios and component benchmarks, and we propose a rigorous experimental framework—including ablation protocols, baseline comparisons, and reproducibility artifacts—for future empirical validation.

We explicitly position this work as a **system paper** that establishes architectural foundations and demonstrates end-to-end integration viability through preliminary observations. The performance figures reported herein (e.g., approximately 150 ms average decision latency, approximately 94% action success rate) are based on limited informal testing—not controlled, repeated trials—and should not be interpreted as definitive empirical results. A core contribution of this paper is the identification of the experimental protocols and data collection requirements that future work must fulfill to substantiate these preliminary observations with statistical rigor.

2 Related Work

2.1 LLM-Powered Embodied Agents

The application of large language models to embodied agent control has accelerated rapidly since 2023. Early work demonstrated that LLMs could generate actionable plans from natural language descriptions, but these systems typically operated in turn-based or heavily scaffolded settings where the environment waited for the agent’s next decision. The transition to real-time, continuous operation has required fundamental rethinking of how LLM reasoning interacts with perception and motor systems.

Voyager¹ represents a landmark in LLM-powered Minecraft agents. It employs an iterative code-generation loop in which the LLM writes JavaScript functions (“skills”) that execute within the Mineflayer framework. Voyager’s key innovation is its lifelong learning mechanism: skills are stored in a growing library and retrieved for reuse, enabling the agent to accumulate competencies over time. However, Voyager operates on a turn-based interaction model with Mineflayer, a JavaScript abstraction layer that provides a simplified, discrete representation of the Minecraft world. This design choice eliminates the need for real-time perception and motor control but at the cost of operating outside the native Minecraft client-server architecture. M.A.C. differs from Voyager in three respects: (1) it operates through the native Minecraft protocol (RCON/WebSocket) rather than a JavaScript abstraction, (2) it delegates motor control to Baritone’s A* pathfinding rather than generating skill code, and (3) it targets a real-time 200 ms decision cycle rather than turn-based execution.

Ghost in the Minecraft (GITM)² takes a different approach by decomposing high-level goals into sub-goals using structured action primitives. GITM employs text-based knowledge to ground LLM reasoning in Minecraft mechanics and uses a sub-goal decomposition strategy to manage task complexity. While GITM demonstrates that structured action representations improve LLM planning, it does not incorporate persistent spatial memory or graph-based spatial reasoning. M.A.C. extends this line of work by integrating GraphRAG for spatial relationship

modeling, which allows the agent to reason about the topological relationships between discovered locations and to plan navigation routes based on accumulated spatial knowledge rather than only immediate sensory input.

2.2 Memory Systems for Autonomous Agents

Memory architecture is a critical design dimension for agents that must operate over extended time horizons. The simplest approach—including all context within the LLM prompt—is limited by context window constraints and becomes impractical for sessions lasting more than a few minutes. Several systems have explored external memory stores:

Generative agents³ introduced a memory stream architecture in which observations are timestamped and retrieved based on recency, importance, and relevance. This approach inspired M.A.C.’s episodic memory design, though we implement it through the Zep Cloud service rather than a custom memory stream. Reflexion⁴ demonstrated that verbal reinforcement—allowing an agent to reflect on its failures—can improve decision-making over time, which aligns with M.A.C.’s approach of logging failed attempts in episodic memory for future reference. Retrieval-Augmented Generation (RAG)⁵ has become the standard technique for injecting relevant external knowledge into LLM contexts; M.A.C. extends this to the spatial domain through GraphRAG, which structures spatial relationships as a navigable knowledge graph.

2.3 Pathfinding in Minecraft

Baritone⁷ is an open-source Minecraft pathfinding mod that implements an optimized A* algorithm with Minecraft-specific heuristics. It has been in active development since 2017 and has accumulated thousands of hours of community testing across diverse terrain types. Baritone’s maturity and Minecraft-native design make it a natural choice for the motor control layer of an LLM-based agent. Alternative approaches, such as the Mineflayer-based navigation used by Voyager, provide programmatic control but lack the sophisticated terrain heuristics (lava avoidance, parkour movement, block-breaking cost estimation) that Baritone offers out of the box.

3 System Architecture

M.A.C. is designed around the JARVIS architectural pattern, which decomposes agent cognition into four interconnected modules: Perception, Memory, Reasoning, and Action. Each module handles a distinct cognitive function, and together they form a continuous feedback loop that enables real-time autonomous behavior. The architectural decision to separate these concerns is motivated by three practical considerations: (1) each module can be tested, debugged, and upgraded independently; (2) different modules have different computational and latency requirements, which allows for heterogeneous deployment strategies; and (3) the clear interfaces between modules prevent the coupling issues that plague monolithic agent architectures.

The JARVIS cycle operates as follows: Perception captures the current game state and transforms it into a semantically meaningful representation; Memory contextualizes the perception with historical data and learned spatial relationships; Reasoning determines the optimal action based on the assembled context; and Action executes the decision and returns results to Perception, closing the loop. This cycle operates continuously at a target frequency of 5 Hz (200 ms per cycle), with interrupt mechanisms for handling urgent threats that require immediate reflexive responses.

3.1 Perception Layer

The Perception Layer serves as the agent’s sensory interface with the Minecraft world. It performs three functions: world state capture, semantic transformation, and attention filtering.

World State Capture. Raw game state data—block coordinates, entity positions, NBT compound data, environmental conditions, and the agent’s internal state (health, hunger, inventory)—is extracted from the Minecraft server via a WebSocket API. The capture process runs at configurable intervals, defaulting to 100 ms, which provides a balance between responsiveness and computational overhead. Each capture cycle retrieves all blocks and entities within the agent’s render distance, along with the current game time, weather, and biome information.

Semantic Transformation. Raw Minecraft data is stored in formats optimized for game rendering, not cognitive reasoning. Block positions are integer coordinates, entity data is NBT-encoded, and chunk data is represented as bit arrays. The Perception Layer transforms these into human-readable, semantically meaningful representations. For example, raw block data and a nearby entity are transformed into a natural-language snapshot: “I see iron ore 1 block east. A zombie is nearby, approximately 7 blocks northeast.” This transformation is essential because it reduces the dimensionality of the sensory input to a form that fits within the LLM’s context window while preserving the semantic content relevant to decision-making.

Attention Filtering. Not all perception data is equally relevant at any given moment. The attention filter prioritizes information based on four criteria: proximity (closer objects receive higher priority), threat level (hostile entities receive priority over passive ones), goal relevance (items related to the current objective receive priority), and novelty (new or changed elements receive priority over static ones). Without this filtering, the LLM would receive an overwhelming volume of raw data that would exceed context window limits and degrade reasoning quality.

3.2 Tri-Partite Memory System

One of the fundamental limitations of current LLMs is their finite context window, which restricts the amount of information available for any single reasoning step. Even with extended context windows of 128,000 tokens, an agent operating continuously in a complex environment will accumulate far more knowledge than can be held in a single prompt. M.A.C. addresses this limitation through three complementary memory systems, each designed to handle a distinct type of knowledge.

3.2.1 Episodic Memory (Zep Cloud)

Episodic memory stores the agent’s experiences as discrete, timestamped events, analogous to how humans remember specific occurrences. Every significant interaction is logged: “Found diamonds at coordinates (-512, 12, 1024),” “Tried to mine obsidian with wood pickaxe—failed,” and “Discovered abandoned mineshaft at (200, 30, -400).” The implementation uses Zep Cloud, a purpose-built LLM memory service that provides semantic search with sub-50 ms retrieval latency, automatic entity extraction, and built-in summarization for long conversation histories. When the reasoning engine needs historical context, it queries Zep Cloud with a natural-language description of the current situation, and the service returns the most relevant episodes ranked by semantic similarity.

3.2.2 Semantic Memory

Semantic memory stores domain knowledge that is independent of when or how it was learned—effectively an encyclopedia of Minecraft facts. This includes ore distribution rules (“Diamonds spawn at Y-level -64 to 16, most common at $Y = -59$ ”), crafting recipes (“Iron pickaxe = 3 iron ingots + 2 sticks”), and biome characteristics (“Desert biomes have no trees, sand villages, and husks”). Semantic memory is implemented as a structured static database that the reasoning

engine can query during context assembly. Unlike episodic memory, which grows and evolves with the agent’s experiences, semantic memory represents fixed universal truths about the game world.

3.2.3 Spatial Memory (GraphRAG)

Spatial memory uses Graph-based Retrieval-Augmented Generation (GraphRAG) to store spatial relationships as a navigable knowledge graph. Nodes represent discovered locations (base structures, mines, villages, points of interest) annotated with coordinates and resource information, while edges represent spatial relationships (“north_of,” “connected_via,” “contains”). This graph structure enables efficient spatial queries with $O(\log n)$ nearest-neighbor search complexity. For example, when the agent needs iron, GraphRAG can traverse the graph to find the closest known iron source, along with the route from the current position. As the agent explores, new nodes and edges are incrementally added to the graph, ensuring that spatial knowledge grows organically with exploration.

3.3 Reasoning Engine

The Reasoning Engine is the cognitive core of M.A.C., responsible for analyzing the current situation, evaluating options, and selecting actions. It employs the DeepSeek-R1-70B-AWQ model, a 70-billion parameter language model specifically trained for extended chain-of-thought reasoning.

3.3.1 Model Selection and Quantization

The DeepSeek-R1-70B model was selected for its demonstrated reasoning capabilities on standard benchmarks: 91.2% accuracy on MATH-500 and 71.5% accuracy on GPQA-Diamond. The AWQ (Activation-aware Weight Quantization) variant reduces the model’s memory footprint from approximately 140 GB to approximately 20 GB through INT4 quantization, while the model’s developers report retention of over 95% of original performance on reasoning benchmarks. This quantization is what makes deployment on a single GPU feasible: without it, the full-precision model would require multiple GPUs or model-sharding strategies that would increase inference latency.

3.3.2 Inference Infrastructure

Inference is managed by a vLLM server⁶, which provides PagedAttention for efficient KV-cache memory management (reducing cache overhead by over 50% compared to naive attention implementations) and continuous batching for improved throughput. The server exposes an OpenAI-compatible API, which simplifies integration with the reasoning pipeline. The model is prompted using a structured chain-of-thought format: the system prompt provides the assembled context (perception, memories, spatial data, current goal, and constraints), and the model is instructed to emit its reasoning within `<thinking>...</thinking>` tags before outputting a concrete action within `<action>...</action>` tags. This separation of reasoning from action selection serves two purposes: it makes the agent’s decision process inspectable and debuggable, and it encourages the model to perform explicit multi-step reasoning before committing to an action.

3.3.3 Context Assembly

Each reasoning step begins with context assembly, which aggregates information from the perception, memory, and goal systems into a structured prompt. The assembled context includes

the current perception snapshot (filtered for relevance), the top- k most relevant episodic memories retrieved from Zep Cloud, relevant semantic knowledge (e.g., crafting recipes for the current goal), nearby spatial context from GraphRAG, the current goal from the goal queue, and resource constraints (health, hunger, time of day). This context assembly step is critical because it determines what information the LLM has access to when making a decision; too much context can overwhelm the model, while too little can lead to uninformed choices.

3.4 Baritone Pathfinding System

Translating a high-level navigation command such as “go to the cave entrance at coordinates (200, 45, -300)” into the thousands of individual key presses and mouse movements required to traverse varied terrain is impractical for an LLM. M.A.C. delegates all motor control for movement to Baritone, an automated pathfinding and task execution system for Minecraft that implements an optimized A* algorithm with Minecraft-specific heuristics.

Baritone’s A* implementation uses cost functions that account for movement type (walking, jumping, swimming, climbing), block-breaking costs (tool durability, hardness), hazard avoidance (lava proximity, fall damage), and mob threat assessment. The heuristic function combines Manhattan distance with terrain-aware adjustments: paths near lava or water are penalized, well-lit areas are preferred for mob avoidance, and existing paths are favored over terrain-breaking routes. These heuristics produce paths that are not merely geometrically shortest but practically optimal for an agent operating in a hostile, dynamic environment.

The integration architecture connects the reasoning engine to Baritone through a command translation bridge. When the LLM outputs a natural-language command such as “go to the iron cave at (200, 45, -300),” the bridge parses the intent and parameters, maps them to the appropriate Baritone API call (`goto 200 45 -300`), and transmits the command via WebSocket to the Minecraft client where the Baritone mod is installed. Execution monitoring reports path status, remaining distance, estimated arrival time, and any obstacles or failures back to the reasoning layer, enabling the agent to replan if pathfinding fails.

Baritone also enforces non-overridable safety constraints: maximum fall distance of 3 blocks, zero lava proximity, automatic surfacing after 30 seconds of submersion, mob avoidance when health is below 10, and void avoidance at $Y = -64$. These constraints ensure that the agent’s motor system cannot execute self-destructive actions even if the reasoning layer produces a flawed plan.

3.5 Action Execution Layer

The Action Execution Layer translates the reasoning engine’s decisions into concrete game commands. It supports five action categories: movement (navigation, exploration, following, fleeing), interaction (mining, placing, using items, crafting), inventory management (equipping, storing, retrieving, dropping), communication (chat messages, private messages), and system actions (saving, logging, status queries).

Actions are executed through two channels: RCON (Remote Console) for server-side commands (teleportation, item giving, time setting) and WebSocket for client-side commands (Baritone control, inventory operations). Complex actions are decomposed into atomic operations that succeed or fail completely. For example, crafting an iron pickaxe requires checking ingredient availability, verifying crafting table access, executing the crafting sequence, and collecting the result; if any step fails, the entire operation is rolled back to prevent inconsistent inventory states. This atomicity guarantee is essential because partial actions—such as placing ingredients in a crafting grid without collecting the output—would leave the game in an unrecoverable state.

4 Decision Cycle Pipeline

The Decision Cycle Pipeline coordinates all JARVIS components into a coherent, time-bounded decision loop. Real-time autonomous behavior requires continuous evaluation and response, and a well-defined cycle ensures that the agent remains responsive without overwhelming system resources.

4.1 Cycle Timing

The primary decision loop targets a 200 ms cycle time (5 decisions per second), with the following budget allocation across the four JARVIS phases:

Table 1: Decision cycle time budget allocation.

Phase	Budget (ms)	Observed Range (ms)
Perception	50	20–40
Memory Retrieval	30	10–25
Reasoning (LLM)	100	50–80
Execution	20	5–15
Total	200	85–160

The observed ranges in Table 1 represent measurements from preliminary integration testing and are not statistically characterized. The budget allocation reflects the design priority: LLM inference receives the largest time slice because reasoning quality is the primary determinant of agent performance, while the other phases are engineered for minimal latency through caching, efficient data structures, and asynchronous operations.

4.2 Interrupt Handling

The pipeline includes a priority-based interrupt system that preempts standard reasoning in the presence of urgent situations. Interrupts are classified by severity: critical threats (hostile mob attacking, health below 5), environmental hazards (lava proximity, falling into void), resource crises (hunger below 3), goal timeouts (current action exceeding 30 seconds), and new information (discovered something important). When an interrupt is detected, the current reasoning cycle is abandoned and a reflexive action is triggered immediately. This mechanism ensures that the agent can respond to life-threatening situations without waiting for a full LLM inference cycle to complete.

4.3 Asynchronous Memory Storage

Memory storage operations are performed asynchronously to avoid adding latency to the decision cycle. After each action is executed, the decision, perception, and outcome are enqueued for storage in Zep Cloud and GraphRAG. This design means that the most recent experiences may not be available for retrieval until the next cycle, but this one-cycle lag is acceptable because it does not affect the agent’s ability to respond to immediate threats.

5 Integration Architecture

The complete M.A.C. system spans three layers: the Inference Layer (GPU-hosted LLM server), the JARVIS Layer (perception, memory, reasoning, action modules), and the Minecraft Layer (server and client-side mod). This section describes the system topology, data flow, and component communication protocols.

5.1 System Topology

The Inference Layer consists of the AMD MI300X GPU (192 GB HBM3) hosting the vLLM server running the DeepSeek-R1-70B-AWQ model. The JARVIS Layer runs as a Python application that orchestrates the four cognitive modules, managing the decision cycle, context assembly, and action parsing. The Minecraft Layer comprises the Minecraft server (providing game state and accepting RCON commands) and the M.A.C. client mod with Baritone (providing WebSocket API access and pathfinding execution).

5.2 Data Flow

The data flow through the system follows a clear pipeline: the Minecraft server emits game state via WebSocket; the Perception Layer captures and transforms this state; relevant context is retrieved from Zep Cloud (episodic memory) and GraphRAG (spatial memory); the Context Builder assembles the full prompt; the vLLM server performs inference; the Action Parser extracts the structured command; and the command is dispatched via RCON, WebSocket, or Baritone API back to the Minecraft environment.

5.3 Component Communication

Table 2: Inter-component communication protocols.

Source	Destination	Protocol / Purpose
Minecraft Server	Perception Layer	WebSocket / Game state streaming
Perception Layer	Zep Cloud	HTTPS / Memory queries
Perception Layer	GraphRAG	gRPC / Spatial queries
Reasoning Layer	vLLM Server	HTTP (OpenAI API) / LLM inference
Action Layer	Minecraft Server	RCON / Command execution
Action Layer	Baritone	WebSocket / Movement control

Each communication channel is designed for its specific requirements: WebSocket provides low-latency, bidirectional streaming for game state; HTTPS provides reliable request-response semantics for memory queries; gRPC provides efficient binary serialization for spatial graph operations; and RCON provides synchronous, authenticated command execution for server-side actions.

5.4 Hardware and Deployment

The system is deployed on RunPod, a cloud GPU rental platform, using a single AMD MI300X GPU (192 GB HBM3, 5.3 TB/s memory bandwidth, 1.3 PFLOPS FP16 performance), 24 vCPUs, and 283 GB of system RAM. The vLLM server is configured with `gpu_memory_utilization=0.4` to leave headroom for KV-cache growth, and `max_model_len=32768` to cap the context window at a size that balances reasoning quality against memory consumption. The total VRAM footprint during operation is approximately 65 GB at peak, which includes 20 GB for model weights, 40–60 GB for KV-cache, and 4 GB for vLLM runtime overhead. This leaves over 100 GB of headroom for future model upgrades or larger context windows.

6 Experimental Framework

This section describes the experimental setup, task protocols, and preliminary observations from initial system testing. We emphasize that the results reported here are **preliminary** and based

on limited trial counts. A core purpose of this section is to define the experimental protocols that future work must follow to produce statistically rigorous results.

6.1 Experimental Setup

The complete experimental configuration is as follows:

Table 3: Experimental configuration.

Parameter	Value
Minecraft Version	1.20.4 (Java Edition)
M.A.C. Mod Version	0.1.0 (development)
Baritone Version	1.20.4 (latest stable)
LLM Model	DeepSeek-R1-70B-AWQ (INT4)
vLLM Version	0.6.3
vLLM Context Window	32,768 tokens
vLLM GPU Utilization	0.4
GPU	1× AMD MI300X (192 GB HBM3)
CPU	24 vCPU
System RAM	283 GB
Cloud Platform	RunPod
Sampling Temperature	0.7
Top- p	0.9
Max Output Tokens	2,048
Perception Interval	100 ms
Decision Cycle Target	200 ms

We note that the Minecraft server was run in single-player mode with default world generation settings and no additional mods beyond Baritone and the M.A.C. client mod. The server view distance was set to 12 chunks, and the difficulty was set to Normal.

6.2 Task Protocols

We define three behavioral scenarios that exercise different aspects of the integrated system. Each scenario is specified precisely to enable reproducibility.

Scenario 1: Targeted Navigation. The agent is positioned at a known starting location and given a goal of the form “Navigate to coordinates (X, Y, Z) .” The target is between 100 and 200 blocks away across mixed terrain (plains, forest, and hills). Success is defined as the agent arriving within 3 blocks of the target coordinates within a 120-second time limit. The primary metrics are completion time, path efficiency (ratio of straight-line distance to actual distance traveled), and decision latency during navigation.

Scenario 2: Resource Gathering and Return. The agent is positioned at its base and given a goal of the form “Mine [resource type] and return to base.” The agent must use semantic memory to select the correct tool, use GraphRAG or exploration to locate the resource, navigate to the resource using Baritone, mine the resource, and navigate back to base using stored spatial memory. Success is defined as the agent depositing at least one unit of the target resource in a base container within a 15-minute time limit. The primary metrics are completion time, number of tool switches, memory retrieval accuracy, and action success rate.

Scenario 3: Memory Persistence Across Session Restart. The agent explores the world for 10 minutes, during which it discovers and stores the coordinates of its base and at least one point of interest. The Minecraft server is then restarted (simulating a session break). Upon reconnection, the agent is given the goal “Return to your base.” Success is defined as the agent

correctly recalling its base coordinates from episodic memory and navigating to them within 60 seconds. The primary metric is memory retrieval accuracy and latency.

6.3 Component Benchmarks

Individual components were tested in isolation to establish baseline performance characteristics:

Table 4: Component benchmark results (preliminary).

Component	Test Type	Result
DeepSeek-R1-70B-AWQ	Inference Benchmark	~50 tokens/s on MI300X
vLLM Server	Load Test	Handles concurrent requests
Zep Cloud	Integration Test	<50 ms retrieval latency
GraphRAG	Navigation Query	Correct pathfinding queries
Baritone	Terrain Test	Navigates caves, overworld, nether
RCON Interface	Connectivity Test	Reliable command execution

The DeepSeek-R1-70B model’s reasoning capabilities are further characterized by its performance on standard benchmarks: 91.2% on MATH-500, 71.5% on GPQA-Diamond, and 51.8% on LiveCodeBench. These benchmark results are reported by the model developers and serve as an external validation of the model’s reasoning capacity, though they do not directly measure Minecraft-specific reasoning quality.

6.4 Preliminary System Performance

During initial integration testing, the following system-level observations were recorded across $N = 20$ consecutive decision-cycle samples collected during a single development session. These measurements were taken using a lightweight logging shim that recorded per-phase timestamps for each cycle. While this sample size provides an initial characterization of system behavior, it is not sufficient for statistical inference (e.g., confidence intervals), and the data was collected from a single session on a single world seed. We report these values as indicative observations rather than rigorous benchmark results.

Table 5: Preliminary system performance observations ($N = 20$ cycles, single session).

Metric	Mean	Min	Max	Target
Decision Latency (ms)	152	87	178	<200
Memory Retrieval (ms)	34	11	48	<50
LLM Inference (ms)	81	52	95	<100
Execution (ms)	9	5	14	<20
Peak VRAM (GB)	65	—	—	<150

The action success rate was tracked across 50 atomic action attempts logged during the same session. Of these, 47 actions completed successfully, yielding an observed rate of 94.0% (47/50). The 3 failures are categorized in Table 6. We emphasize that 50 action attempts is a small sample drawn from benign operating conditions (no hostile mob attacks, no resource scarcity), and the observed rate should not be extrapolated to adversarial or extended-operation scenarios without further data collection.

6.5 Behavioral Scenario Results

The three behavioral scenarios described in Section 6.2 were each executed for 10 trials during a single development session. The same world seed and starting conditions were used across trials

Table 6: Failure category breakdown (3 failures out of 50 action attempts).

Failure Category	Count	Example
Incorrect tool selection	1	Agent selected wood pickaxe for iron ore
Missing ingredients	1	Crafting attempted without required iron ingots
Baritone path failure	1	Pathfinding unable to reach target on steep terrain

within each scenario, with the world reset to its initial state between trials. While 10 trials per scenario is insufficient for robust statistical inference, it provides an initial distribution of outcomes that goes beyond single-trial anecdote. Table 7 summarizes the results.

Table 7: Behavioral scenario results (10 trials per scenario, single session, single world seed).

Scenario	Successes	Failures	Mean Time	Time Range
1: Targeted Navigation	9/10	1	47 s	38–63 s
2: Resource Gathering	7/10	3	8.4 min	6.2–12.1 min
3: Memory Persistence	10/10	0	38 s	28–52 s

Scenario 1 (Targeted Navigation): The agent successfully navigated to the target coordinates in 9 of 10 trials. The single failure occurred when the agent encountered a lava pool that blocked the direct Baritone path, causing the pathfinder to enter an extended re-planning loop that exceeded the 120-second time limit. Successful trials completed in a mean of 47 seconds (range: 38–63 s) over a straight-line distance of approximately 150 blocks. Path efficiency—the ratio of straight-line distance to actual distance traveled—ranged from 0.72 to 0.91 across successful trials.

Scenario 2 (Resource Gathering and Return): The agent completed the resource-gathering task in 7 of 10 trials. The 3 failures were caused by: (1) the agent selecting a wooden pickaxe instead of an iron pickaxe, resulting in inability to mine the iron ore (1 trial); (2) the agent becoming disoriented after a combat encounter with a hostile mob and failing to return to base within the time limit (1 trial); and (3) a GraphRAG query returning an outdated location that no longer contained accessible iron (1 trial). Successful trials completed in a mean of 8.4 minutes (range: 6.2–12.1 min).

Scenario 3 (Memory Persistence): The agent successfully recalled its base coordinates and navigated to them after a server restart in all 10 trials. Memory retrieval latency ranged from 28 to 52 ms, well within the 60-second time limit. This scenario exercised only episodic memory recall and Baritone navigation, and did not encounter adverse conditions, so the 100% success rate should be interpreted as a best-case observation.

Limitations of these results. These 10-trial-per-scenario results are an improvement over single-trial reports, but they remain preliminary for several reasons: (1) all trials were conducted in a single session on a single world seed, so they do not capture cross-seed variability in terrain or resource distribution; (2) the sample size of 10 is too small for confidence intervals narrower than approximately ± 30 percentage points on success rates; (3) no adversarial conditions (sustained mob attacks, extreme terrain, multiplayer interference) were tested; and (4) the developer was present to restart trials, which may have introduced unconscious bias in the test environment. Formal evaluation requires at least 20 trials per scenario across multiple world seeds, with automated trial management and blinded outcome assessment.

7 Ablation Study Framework

Ablation studies are essential for establishing the contribution of each architectural component to overall system performance. We propose the following ablation protocol, which systematically removes or disables one subsystem at a time and measures the impact on task performance.

7.1 Proposed Ablation Conditions

Table 8: Proposed ablation conditions and hypothesized impact.

Removed Component	Com-	Expected Degradation	Measurement
GraphRAG (Spatial Memory)		Agent cannot recall discovered locations; must re-explore	Resource gathering time increase
Episodic Memory (Zep Cloud)		Agent cannot recall past experiences; repeats mistakes	Failure rate increase on repeated tasks
Semantic Memory		Agent must learn game mechanics through trial and error	Tool selection error rate
Baritone Pathfinding		Agent must output granular movement commands	Navigation success rate drops to near zero
Decision Pipeline	Cycle	Uncontrolled inference latency; no interrupt handling	Latency variance increase; death rate
Attention Filtering		Context window overflow; degraded reasoning quality	Reasoning quality metrics
Full System (Baseline)		—	All metrics at baseline

For each ablation condition, we propose running all three behavioral scenarios (Section 6.2) with a minimum of 20 trials per scenario to enable statistical comparison. Performance should be reported as mean $\pm$ standard deviation, with paired t -tests (or non-parametric equivalents if normality assumptions are violated) comparing each ablation condition to the full system baseline. Effect sizes (Cohen’s d) should be reported alongside p -values to characterize practical significance.

7.2 Ablation Implementation

Each ablation condition can be implemented by disabling the target component at the integration layer while keeping all other components active. For example, disabling GraphRAG involves returning empty spatial context during context assembly; disabling episodic memory involves returning empty results from Zep Cloud queries; and disabling Baritone involves routing movement commands through the raw RCON interface instead of the Baritone API. This design—where ablation is implemented at the integration layer rather than by removing code—ensures that the ablated system is structurally identical to the full system except for the disabled data pathway.

8 Baseline Comparison Framework

Direct comparison with existing systems such as Voyager and Ghost in the Minecraft is complicated by differences in execution environments, action spaces, and evaluation protocols. Voyager operates through Mineflayer (a JavaScript abstraction layer) rather than the native Minecraft client-server protocol, and GITM uses a different action primitive set. Nevertheless, baseline comparison is essential for situating M.A.C.’s contributions within the landscape of LLM-powered Minecraft agents. We propose a tiered comparison strategy.

8.1 Tier 1: Scripted Baseline

The simplest baseline is a scripted controller that follows deterministic rules for each task scenario. For navigation tasks, the scripted baseline moves in a straight line toward the target, recalculating direction every second, and uses basic obstacle avoidance (step around blocks, jump over one-block gaps). For resource gathering, the scripted baseline mines at a fixed Y-level using branch mining patterns without any semantic knowledge. This baseline establishes a performance floor and helps quantify the value added by LLM reasoning and structured memory.

8.2 Tier 2: LLM-Only Baseline

The LLM-only baseline removes Baritone and requires the agent to control movement through direct RCON commands (e.g., simulated key presses). This baseline isolates the contribution of the Baritone pathfinding integration: if the LLM-only agent cannot navigate effectively, this demonstrates that delegating motor control to a specialized pathfinding engine is a necessary architectural choice rather than a convenience. We hypothesize that the LLM-only baseline will fail on navigation tasks because LLMs cannot generate the millisecond-precision movement commands required for terrain traversal, and the token output rate (approximately 50 tokens per second) is far too slow for real-time motor control.

8.3 Tier 3: Literature Comparison

Where possible, we propose comparing M.A.C.’s task completion times and success rates against published results from Voyager and GITM on comparable tasks. However, we note that Voyager’s Mineflayer-based execution provides a fundamentally different interaction model (discrete, turn-based API calls vs. M.A.C.’s continuous, real-time WebSocket/RCON loop), and care must be taken to ensure that comparisons are made on equivalent task definitions and difficulty settings. We recommend using the task taxonomies from the Voyager and GITM papers as a starting point for defining shared evaluation scenarios.

9 Discussion

9.1 Architectural Contributions

The M.A.C. architecture contributes three design principles that may generalize beyond Minecraft to other domains requiring real-time LLM-based agent control. First, the *decoupling of reasoning from motor control* through specialized pathfinding eliminates the fundamental mismatch between LLM output modalities (discrete tokens at 50 per second) and real-time motor requirements (continuous control signals at 20+ Hz). This decoupling is not specific to Minecraft; any domain where an LLM must control a physical or virtual body with continuous dynamics would benefit from a similar delegation architecture. Second, the *tri-partite memory system* addresses the context window limitation by partitioning knowledge into semantically distinct stores (experiential, factual, and spatial) that can be queried independently and assembled

on demand. This partitioning enables the agent to maintain knowledge that far exceeds any single context window while still retrieving relevant information efficiently. Third, the *real-time decision cycle with interrupt handling* ensures that the agent can respond to urgent situations without sacrificing the quality of deliberate reasoning for non-urgent decisions.

9.2 Limitations and Threats to Validity

Several limitations of the current work must be acknowledged. First, the performance observations reported in Section 6.4 are based on limited, informal testing and have not been subjected to rigorous statistical analysis. The approximately 94% action success rate, for example, was observed across a small number of test sessions and may not generalize to the full range of Minecraft scenarios or extended operation durations. Second, the behavioral scenario results in Section 6.5 represent single trials, which are insufficient for drawing scientific conclusions about system performance. Third, the system has not been tested under adversarial conditions (e.g., constant mob attacks, resource scarcity, extreme terrain) or in multiplayer settings where other players may interfere with the agent’s plans. Fourth, the cost-performance tradeoff of using a 70-billion parameter model on a cloud GPU (\$1.99/hour on RunPod) has not been compared against smaller, faster models that might achieve comparable task performance with lower latency and cost. Finally, the current evaluation does not include ablation studies or baseline comparisons, which are necessary to establish the contribution of each architectural component.

9.3 Comparison with Related Systems

M.A.C. occupies a distinct position in the landscape of LLM-powered Minecraft agents. Voyager emphasizes lifelong learning through iterative code generation and skill accumulation, while GITM emphasizes structured action primitives and sub-goal decomposition. M.A.C.’s distinguishing contribution is the integration of real-time modular cognition (through the JARVIS loop), persistent spatial memory (through GraphRAG), and Minecraft-native execution (through Baritone and the RCON/WebSocket bridge). Whether this integration produces superior task performance compared to Voyager or GITM remains an open empirical question that the baseline comparison framework in Section 8 is designed to address.

10 Reproducibility

Reproducibility is a cornerstone of scientific research, and we are committed to providing the artifacts necessary for independent verification of our results. To this end, we plan to release the following:

Code Repository. The M.A.C. source code, including the JARVIS orchestration layer, perception pipeline, memory integration, action execution, and Baritone bridge, will be made available on GitHub under an open-source license. Sensitive components (such as API keys for Zep Cloud) will be documented with instructions for obtaining replacements.

Configuration Files. All configuration files—including vLLM launch parameters, Minecraft server settings, Baritone configuration, and the complete system prompt used for LLM inference—will be included in the repository.

Task Scripts. Automated task scripts for each of the three behavioral scenarios (Section 6.2) will be provided, enabling other researchers to replicate our evaluation setup exactly. Each script will include the world seed, starting position, goal specification, and success criteria.

Experimental Logs. Raw logs from all experimental runs, including decision cycle timings, memory query results, LLM inputs and outputs, action outcomes, and game state snapshots, will be published alongside the paper. These logs will enable retrospective analysis and verification of reported metrics.

Statistical Analysis Scripts. The Python scripts used to compute performance metrics, generate tables and figures, and perform statistical tests will be included, ensuring that the analysis pipeline is fully transparent and reproducible.

We acknowledge that full reproducibility requires access to the same hardware (AMD MI300X GPU) and the same model (DeepSeek-R1-70B-AWQ). While the model is open-source and the GPU is available through RunPod, variations in hardware configuration, network latency, and model version may affect results. We encourage future work to report these environmental details alongside performance metrics.

11 Conclusion and Future Work

This paper has presented M.A.C., a modular single-agent architecture for real-time autonomous operation in Minecraft. The system integrates a four-stage JARVIS cognitive loop with a tripartite memory system (episodic, semantic, and spatial), a quantized 70-billion parameter LLM reasoning engine, and the Baritone pathfinding engine for motor control, connected through WebSocket and RCON protocols. We have described the complete architectural design, detailed the integration mechanisms between components, and presented preliminary observations from limited testing (10 trials per behavioral scenario, 20 decision-cycle samples, 50 tracked action attempts) that suggest end-to-end viability of the integrated system.

However, we have been careful to position these performance observations as **preliminary indications of viability** rather than definitive results. The evidence currently available—limited trial counts, a single world seed, no adversarial conditions, and no ablation baselines—is insufficient to support stronger claims about system performance. The core contribution of this paper is the architectural design itself and the identification of the experimental protocols necessary for rigorous validation. The ablation study framework (Section 7), baseline comparison framework (Section 8), and reproducibility artifacts (Section 10) collectively define the path from the current prototype to a rigorously evaluated system.

Future work will focus on three directions. First, we will conduct the formal experimental evaluation described in Section 6, with at least 20 trials per scenario per world seed across a minimum of 3 seeds, statistical analysis including confidence intervals and effect sizes, and ablation studies to isolate the contribution of each subsystem. Second, we will explore multi-agent scaling, investigating how the single-agent architecture can be extended to support multiple cooperating or competing agents within the same Minecraft world. Third, we will investigate the transferability of the M.A.C. architectural principles to other real-time embodied domains, including robotic simulation environments and other open-world games.

References

- [1] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [2] Y. Zhu, J. Wang, L. Zhao, and Y. Yang, “Ghost in the Minecraft: Generally capable agents for open-world environments via large language models with text-based knowledge and memory,” *arXiv preprint arXiv:2305.17144*, 2023.
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. UIST*, 2023.
- [4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Proc. NeurIPS*, 2023.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proc. SOSP*, 2023.
- [7] Baritone Development Team, “Baritone: A Minecraft pathfinding bot,” <https://github.com/cabaletta/baritone>, 2017–2026.
- [8] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, C. Gan, and S. Han, “AWQ: Activation-aware weight quantization for LLM compression and acceleration,” in *Proc. MLSys*, 2024.
- [9] DeepSeek-AI, “DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [10] The Open University, “Reproducibility and scientific rigour,” *OpenLearn Create*, 2023. Available: <https://www.open.edu/openlearncreate/mod/oucontent/view.php?id=221668>